

03. live_wifi ROS2 launch 入口

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	4
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 launch Action、Node Action、実行可能名、remapping	6
6.7 天候、route、補間、時刻、単位	7
6.8 class、独自クラス、インスタンス、self、継承	7
6.9 rclpy、rcl、rmw、DDS/RTPS の層	8
6.10 ROS graph、Node、topic、service、action、parameter	8
6.11 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか	8
7 関数・クラスを上から順に解説	9
7.1 L12 関数 _setup	9
7.2 L62 関数 generate_launch_description	10

7.3 launch から起動するノード	11
8 処理の流れ	11
9 このファイルの位置づけ	11

1 対象

項目	内容
ファイル	launch/solar_race_live_wifi.launch.py
ソース SHA-256	e4397855dd5375061242cb52be543a5fb2602f5936a7f661dddad529996d2491
種別	ROS 2 launch Python
区分	launch
役割	WiFi テレメトリ、風補正、live MPC 運用をまとめて起動する launch 入口。
起動文脈	live_wifi モード起動時に ros2 launch される。

2 このファイルを読む理由

WiFi テレメトリ、風補正、live MPC 運用をまとめて起動する launch 入口。

- profile_yaml を launch 引数として受ける。
- 共通 live node 群に WiFi bridge と wind correction を追加する。
- live forecast の raw/corrected CSV の入口でもある。

3 呼び出し関係

3.1 どこから呼ばれるか

- scripts/solar_control.sh
- SolarSim.ps1

3.2 次に読むと理解が進むファイル

- mpc_solarcar/live_launch.py
- mpc_solarcar/telemetry_text_bridge_node.py
- mpc_solarcar/wind_correction_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/live_launch.py
- mpc_solarcar/solar_profile.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
nodes = build_live_nodes(profile_path, cfg, use_wifi=True)
if bool(wifi_cfg.get("enabled", True)):
    nodes.append(
        Node(
            package="mpc_solarcar",
            executable="telemetry_text_bridge_node",
            name="telemetry_text_bridge_node",
            parameters=[
                {
                    **{key: value for key, value in wifi_cfg.items() if key != "enabled"},
                    "solar_power_gain_to_pack": float(
                        model_cfg.get("solar_measurement_gain_to_pack", 1.0)
                    ),
                },
            ],
        ),
    )
```

5 主要構造の読み取り

主要関数は `generate_launch_description`。launch から起動する Node action は 2 件。

5.1 主要クラス・関数

行	種別	名前	補足
12	function	<code>_setup</code>	args: context
62	function	<code>generate_launch_description</code>	

5.2 ファイルを上から読んだときの定義順

行	内容
12	関数 <code>_setup</code> を定義する。
62	関数 <code>generate_launch_description</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future__ import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>from launch import LaunchDescription</code>	launch が実行すべき action 群をまとめるため。このファイル内での主な使用位置は L63。
4	<code>from launch.actions import DeclareLaunchArgument, OpaqueFunction</code>	DeclareLaunchArgument や OpaqueFunction など launch action を使うため。このファイル内での主な使用位置は L65, L66。
5	<code>from launch.substitutions import LaunchConfiguration</code>	launch 引数の実行時値を参照するため。このファイル内での主な使用位置は L13。

6	<code>fromlaunch_ros. actionsimportNode</code>	ROS 2 ノード起動 action を launch から記述するため。 このファイル内での主な使用位置は L25, L41。
8	<code>frommpc_solarcar.live_ launchimportbuild_live_ nodes, live_forecast_paths</code>	live 系 node 構成ビルダから <code>build_live_nodes</code> , <code>live_forecast_paths</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/live_launch.py</code> 。このファイル内での主な使用位置は L20, L22。
9	<code>frommpc_solarcar.solar_ profileimportget_section, load_profile</code>	profile YAML 読込と検証から <code>get_section</code> , <code>load_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L14, L15, L16, L17, L18。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。`'x = f()'` では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

`'float(...)'`、`'int(...)'`、`'bool(...)'` は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

`'None'` は値が無いことを示す単一のオブジェクト、`'math.nan'` は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、`'is None'` と `'math.isfinite'` を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 launch Action、Node Action、実行可能名、remapping

‘launch_ros.actions.Node(...)’は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

‘DeclareLaunchArgument’は launch 実行時に受け取る入力欄を宣言し、‘LaunchConfiguration’はその値を後で解決する substitution を表す。‘perform(context)’は実行時 context から確定文字列を取り出す。

launch の ‘name’は Node 名 override、‘parameters’は起動時 parameter、‘remappings’は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

6.7 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.8 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.9 rclpy、rcl、rmw、DDS/RTSPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC 内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executorの実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.10 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.11 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_kmh /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

7 関数・クラスを上から順に解説

7.1 L12 関数 `_setup`

- 行範囲: L12–L59
- 所属: module 直下
- 定義: `_setup(context)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `LaunchConfiguration`, `Node`, `append`, `bool`, `build_live_nodes`, `float`, `get`, `get_section`, `items`, `live_forecast_paths`, `load_profile`, `perform`
- 戻り値の要点: `nodes`
- 主なローカル代入名: `_base_csv`, `cfg`, `corrected_csv`, `key`, `live_cfg`, `model_cfg`, `nodes`, `profile_path`, `profile_yaml`, `raw_forecast_csv`, `value`, `wifi_cfg`, `wind_cfg`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `profile_yaml` に `LaunchConfiguration('profile_yaml').perform(context)` の結果を代入する。
2. `(profile_path, cfg)` に `load_profile(profile_yaml)` の結果を代入する。
3. `live_cfg` に `get_section(cfg, 'live')` の結果を代入する。
4. `wifi_cfg` に `get_section(live_cfg, 'wifi_bridge')` の結果を代入する。
5. `wind_cfg` に `get_section(live_cfg, 'wind_model')` の結果を代入する。
6. `model_cfg` に `get_section(cfg, 'model')` の結果を代入する。
7. `(_base_csv, raw_forecast_csv, corrected_csv)` に `live_forecast_paths(profile_path, cfg)` の結果を代入する。
8. `nodes` に `build_live_nodes(profile_path, cfg, use_wifi=True)` の結果を代入する。

9. 条件 `bool(wifi_cfg.get('enabled', True))` を判定し、真なら内部処理を行う。
10. `nodes.append(...)` を実行する。
11. 条件 `bool(wind_cfg.get('enabled', True))` を判定し、真なら内部処理を行う。
12. `nodes.append(...)` を実行する。
13. `nodes` を返す。

代表コード断片

```
def _setup(context):
    profile_yaml = LaunchConfiguration("profile_yaml").perform(context)
    profile_path, cfg = load_profile(profile_yaml)
    live_cfg = get_section(cfg, "live")
    wifi_cfg = get_section(live_cfg, "wifi_bridge")
    wind_cfg = get_section(live_cfg, "wind_model")
    model_cfg = get_section(cfg, "model")

    _base_csv, raw_forecast_csv, corrected_csv = live_forecast_paths(profile_path, cfg)

    nodes = build_live_nodes(profile_path, cfg, use_wifi=True)
    if bool(wifi_cfg.get("enabled", True)):
        nodes.append(
            Node(
                package="mpc_solarcar",
                executable="telemetry_text_bridge_node",
                name="telemetry_text_bridge_node",
                parameters=[
                    {
                        **{key: value for key, value in wifi_cfg.items() if key != "enabled"},
                        "solar_power_gain_to_pack": float(
                            model_cfg.get("solar_measurement_gain_to_pack", 1.0)
                        ),
                    },
                ],
            )
        )
    if bool(wind_cfg.get("enabled", True)):
        nodes.append(
            Node(
                package="mpc_solarcar",
                executable="wind_correction_node",
                name="wind_correction_node",
                parameters=[
                    {

```

7.2 L62 関数 `generate_launch_description`

- 行範囲: L62–L68
- 所属: `module` 直下
- 定義: `generate_launch_description()`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DeclareLaunchArgument`, `LaunchDescription`, `OpaqueFunction`

- 戻り値の要点: `LaunchDescription([DeclareLaunchArgument('profile_yaml', default_value='config/solar/bwsc_2027_demo.yaml'), OpaqueFunction(function=_setup)])`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `LaunchDescription([DeclareLaunchArgument('profile_yaml', default_value='config/solar/bwsc_2027_demo.yaml'), OpaqueFunction(function=_setup)])` を返す。

代表コード断片

```
def generate_launch_description():
    return LaunchDescription(
        [
            DeclareLaunchArgument("profile_yaml", default_value="config/solar/bwsc_2027_demo
.yaml"),
            OpaqueFunction(function=_setup),
        ]
    )
```

7.3 launch から起動するノード

行	executable	package / name
25	telemetry_text_bridge_node	package=mpc_solarcar, name=telemetry_text_bridge_node
41	wind_correction_node	package=mpc_solarcar, name=wind_correction_node

8 処理の流れ

1. `launch` 引数を受け取り、`profile` を読み込む。
2. `Node action` を動的に組み立てる。
3. `LaunchDescription` として ROS 2 `launch` へ返す。

9 このファイルの位置づけ

この資料群では、`launch/solar_race_live_wifi.launch.py` を **launch** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。